multi_brains Framework Source Code Analysis

multi_brains Framework Source Code Analysis

1. Course Content
2. Source Code Package Structure
   - 2.1 Package File Structure
3. Speech Recognition Functionality
   - 3.1 Dynamic Recording with Voice Activity Detection
   - 3.2 ASR Speech Recognition
4. Model Service Functionality
5. Action Server Functionality
6. Interrupt Functionality
   - 6.1 Interrupting During Recording
   - 6.2 Interrupting During Conversation
   - 6.3 Interrupting During Action
     - 6.3.1 Normal Action Interruption
     - 6.3.2 Interruption of Actions with Subprocesses

1. Course Content

- Analyze the core functions of the multi_brains package architecture.
- In subsequent lessons, new action functions introduced in each tutorial will be explained separately.

> [!TIP]
>
> - This lesson is a pure code analysis course, intended for users who want an in-depth understanding of the functionality. If you just want to experience the features, you can skip this lesson.

2. Source Code Package Structure

2.1 Package File Structure

```
├── config
│   ├── map_mapping.yaml
│   ├── multi_brains_setting.yaml
│   └── README.MD
├── language
│   ├── en.yaml
│   └── zh.yaml
├── launch
│   └── llm_agent_control.launch.py
├── multi_brains
│   ├── action_service.py
│   ├── asr_detect.py
│   ├── __init__.py
│   ├── model_service.py
│   └── utils
├── package.xml
```

```
├── resource
│   └── multi_brains
├── setup.cfg
├── setup.py
├── system_vioce
│   ├── en
│   ├── notify.mp3
│   ├── test_en.wav
│   ├── test_zh.wav
│   └── zh
└── test
    ├── test_copyright.py
    ├── test_dify_connection.py
    ├── test_flake8.py
    ├── test_pep257.py
    ├── test_PiperTTS.py
    ├── test_SenseVoiceSmall.py
    ├── test_TongyiASR.py
    ├── test_TongyiTTS.py
    ├── test_xunfeiASR.py
    └── test_xunfeiTTS.py
```

- **config**

  Configuration folder, stores configuration **file templates**.

- **multi_brains**

  Source code folder.

- **asr_detect.py**

  Speech recognition program file.

- **model_service.py**

  Model server program file, used to call various model interfaces to implement the model inference architecture.

- **action_service.py**

  Action server program file, used to receive the action list requested by the model server and control the robot's movement.

- **launch**

  Launch file folder, used to store ROS2 node launch files.

- **language**

  Language pack, stores log files for different languages.

- **system_voice**

  System audio files.

3. Speech Recognition Functionality

Source Code Path:

```
~/yahboomM3_ws/code_ws/src/multi_brains/multi_brains/asr_detect.py
```

3.1 Dynamic Recording with Voice Activity Detection

1. Continuously read audio frames and perform voice activity detection.
2. If speech is detected, add the audio frames to a buffer; if continuous silence is detected beyond a threshold (90 frames, approx. 1.5s), stop recording. The tail-end detection duration can be modified via the parameter file.
3. After recording, remove the trailing silence and save the valid speech as a WAV file.
4. If no valid speech is detected, the file is not saved.

```
    def listen_for_speech(self):
        '''Dynamic recording with VAD'''
        self.record_flag = True
        PRE_SPEECH_FRAMES  = 5         # 150ms speech start compensation
        PRINT_EVERY_N_FRAMES = 5

        recording_active = False
        silence_counter  = 0
        print_counter    = 0

        audio_buffer = []
        pre_speech_buffer = deque(maxlen=PRE_SPEECH_FRAMES)
        stream = self.pyaudio.open(
            format=pyaudio.paInt16,
            channels=1,
            rate=self.sample_rate,
            input=True,
            frames_per_buffer=self.frame_bytes,
        )
        self.play_audio(self.notify_audio)
        try:
            while not self.stop_event.is_set():
                frame = stream.read( self.frame_bytes,
exception_on_overflow=False)
                is_speech = self.vad.is_speech(frame, self.sample_rate)
                print_counter += 1
                if print_counter >= PRINT_EVERY_N_FRAMES:
                    print("1-1-1" if is_speech else "-----")
                    print_counter = 0
                if not recording_active:
                    # ---- IDLE ----
                    pre_speech_buffer.append(frame)
                    if is_speech:
                        #  RECORDING
                        recording_active = True
                        audio_buffer.extend(pre_speech_buffer)
                        pre_speech_buffer.clear()
                        audio_buffer.append(frame)
                        silence_counter = 0
                else:
                    # ---- RECORDING ----
                    audio_buffer.append(frame)
                    if is_speech:
                        silence_counter = max(0, silence_counter - 1)
                    else:
                        silence_counter += 1
                        if silence_counter >= self.MAX_SILENCE_FRAMES:
```

```
                            break
        finally:
            stream.stop_stream()
            stream.close()
            self.record_flag = False

        if recording_active and audio_buffer:
            with wave.open(self.user_speech_dir, "wb") as wf:
                wf.setnchannels(1)
                wf.setsampwidth(self.pyaudio.get_sample_size(pyaudio.paInt16) )
                wf.setframerate(self.sample_rate)
                wf.writeframes(b"".join(audio_buffer))
            return True
        return False
```

3.2 ASR Speech Recognition

- Calls the `recognize` method of the speech engine for speech recognition. The speech engine is provided by instantiated objects of the `Tongyi_ASR`, `SenseVoiceSmall_ASR`, and `XUNFEI_ASR` classes.

```
    def kws_handler(self) -> None:
        '''Wake-up handling function'''
        if self.stop_event.is_set():
            self.logger.info("Wake-up processing thread interrupted, no handle
user speech.!!!!!")
            return
        if self.listen_for_speech():
            asr_result=self.asr_engine.recognize(self.user_speech_dir)  #
Perform ASR conversion
            if not asr_result[0]:
                self.logger.error(Fore.RED+f"Speech recognition failed because
the audio segment is empty or the speech model is unavailable. ASR_OUT:
{asr_result[1]}"+Fore.RESET)
            else:
                if len(asr_result[1]) > self.asr_threashold:
                    self.logger.info(Fore.GREEN+"ASR Result:
"+asr_result[1]+Fore.RESET)

 self.asr_result_queue.put([asr_result[1],'text_request',False]) # Put the ASR
result into the queue
                else:
                    self.logger.info(Fore.YELLOW+"The voice recognition result
is too short. Could it be that the user woke it up by mistake
"+asr_result[1]+Fore.RESET)
```

4. Model Service Functionality

Source Code Path:

```
~/yahboomM3_ws/code_ws/src/multi_brains/multi_brains/model_service.py
```

Core Function Analysis:

- Checks the `llm_handler_queue` in a separate thread for model access requests.
- Once an element is pushed to the queue, it calls the `Dify` access interface `chat` function with different parameters based on the request type.
- After receiving the response from the Dify AI agent, it parses the content for speech playback and sends the action list to the action server for execution.

```
    def handle_llm_thread(self)->None:
        '''
        Handle model request
        '''
        while True:
            if not self.llm_handler_queue.empty(): # If the queue is not empty,
process the model request
                if not self.text_chat_mode and self.asr_detect.record_flag:
continue
                request_query = self.llm_handler_queue.get()
                if self.debug_mode: self.get_logger().info(f"Processing LLM
request: {request_query}")

                if request_query[1] == 'text_request':
                    '''text request'''
                    result = self.dify_llmclient.chat(request_query[0],
robot_feedback=request_query[2])
                elif request_query[1] == 'image_request':
                    '''vision + text request'''
                    result = self.dify_llmclient.chat(request_query[0],
image_path=self.image_cache_path, robot_feedback=request_query[2])

                if result[0]:
                    if not self.text_chat_mode and self.asr_detect.record_flag:
continue
                    split_result = self.extract_actions(result[1])
                    if split_result is None: continue
                    action_list, llm_response, decision_plan =
self.extract_actions(result[1])
                    if decision_plan is not None:
                        self.get_logger().info(Fore.YELLOW +
self.syslog.get_text("system_log_3", decision_plan=decision_plan) + Fore.RESET)
                    self.get_logger().info(Fore.YELLOW + f'"action":
{action_list},"response": {llm_response}' + Fore.RESET)

                    if not self.text_chat_mode: # Voice response
                        if self.tts_engine.synthesize(llm_response,
self.tts_out_path):
                            self.play_audio(self.tts_out_path)
                        else:
                            self.get_logger().error(Fore.RED + "Speech synthesis
failed. Check whether the TTS model is available" + Fore.RESET)
                    else: # Text response
                        if decision_plan is not None:

 self.text_pub.publish(String(data=self.syslog.get_text("system_log_3",
decision_plan=decision_plan)))
                        self.text_pub.publish(String(data=f'"action":
{action_list}, "response": {llm_response}'))
```

```
                    if action_list != []: self.send_action_service(action_list,
llm_response)
                else:
                    self.get_logger().error(Fore.RED + f"The model request
failed. Check whether Dify or the AI model is normal.\
                                            Error Log:{result[1]}" + Fore.RESET)
            else:
                time.sleep(1.0) # Sleep for 1 second when there are no requests
```

5. Action Server Functionality

Source Code Path:

```
~/yahboomM3_ws/code_ws/src/multi_brains/multi_brains/action_service.py
```

- Contains the implementation of all basic action functions the robot can perform. It receives action list requests and executes the corresponding actions. It also has the ability to be interrupted upon re-awakening.

Core Callback Function `execute_callback` Analysis:

- Accepts a string containing the list of actions.

```
    def execute_callback(self, goal_handle):
        """action execution callback function"""
        actions = goal_handle.request.actions
        feedback_result = None
        if self.debug_mode:
self.get_logger().info(self.actionlog.get_text("debug_log_1", actions=actions))
        self.action_runing = True
        for action in actions:
            if self.interrupt_event.is_set():
                break
            match = re.match(r"(\w+)\((.*)\)", action)
            action_name, args_str = match.groups()
            args = [arg.strip() for arg in args_str.split(",")] if args_str else
[]
            if not hasattr(self, action_name):
                self.get_logger().error(Fore.RED + f"action_service: {action} is
an invalid action, skipping execution" + Fore.RESET)
            else:
                method = getattr(self, action_name)
                feedback_result = method(*args)

        if not self.interrupt_event.is_set(): # Feedback action execution result
to dify-agent
            msg = LlmRequest()
            if feedback_result == False:
                # Action execution failed
                msg.llm_request = self.actionlog.get_text("action_feedback_2",
action_name=actions)
                msg.robot_feedback = True
                self.llm_request_pub.publish(msg)
            elif feedback_result == True:
```

```
                # Action execution successful
                msg.llm_request = self.actionlog.get_text("action_feedback_1",
action_name=actions)
                msg.robot_feedback = True
                self.llm_request_pub.publish(msg)
            elif feedback_result == None:
                # No feedback for null operation
                if self.debug_mode:
self.get_logger().info(self.actionlog.get_text("system_log_1"))

            if self.debug_mode: self.get_logger().info(msg.llm_request)

        if self.debug_mode:
self.get_logger().info(self.actionlog.get_text("system_log_2"))
        self.action_runing = False
        self.interrupt_event.clear()
        goal_handle.succeed()
        result = Rot.Result()
        result.success = True
        return result
```

6. Interrupt Functionality

The robot supports interrupting at any stage, which can be divided into recording, conversation, and action stages. The principles of interruption for each stage are described here.

6.1 Interrupting During Recording

If you misspeak during the recording process or are not satisfied with the content and want to re-record, you can directly wake up the robot again **during the recording** to interrupt the previous recording and start a new one.

- The logic is implemented in the `asr_detect_run` method in the `asr.py` file:
- Each time the robot is woken up, if there is already a running recording thread, it is interrupted using the `stop_event` thread event, and the system waits for it to finish.
- After clearing the stop event flag, a new recording thread is started.

```
    def asr_detect_run(self):
        while True:
            # Process only the most recent wake-up request to prevent duplicates
            if self.wakeup_event.wait(timeout=0.1):
                self.wakeup_event.clear()
                self.extern_wakeup.set()
                self.publisher.wakeup_pub.publish(Bool(data=True))

                self.logger.info("I'm here💥")
                self.wake_up_voice() # Respond to the user
                if self.current_thread and self.current_thread.is_alive():   #
Interrupt the previous wake-up handling thread
                    self.stop_event.set()
                    self.current_thread.join()  # Wait for the current thread to
finish
                    self.stop_event.clear()  # Clear the event
                self.current_thread = threading.Thread(target=self.kws_handler)
```

```
                self.current_thread.daemon = True
                self.current_thread.start()
            time.sleep(0.5)
```

6.2 Interrupting During Conversation

If you are not satisfied with the robot's response while it is speaking or do not want it to continue, you can use the wake-up word to interrupt it and start recording your speech immediately. You can then give the robot a new command (within the current task cycle) or say "End current task" to terminate the current task and start a new one.

- The logic is implemented in the `wakeup_callback` and `play_audio` methods of the **CustomActionServer** class in the `action_service.py` file:
- `wakeup_callback` is the callback function for handling wake-ups. Each time the robot is woken up in `asr.py`, a wake-up signal is published via topic communication, which `wakeup_callback` subscribes to and processes.
- Each time it wakes up, it checks if **pygame.mixer** is playing audio. If so, it notifies the playback thread to stop via the `self.stop_event` thread event.
- If a previous action is still running after wake-up, the **self.interrupt_flag** is set for subsequent action interruption.

```
def wakeup_callback(self, msg):
    if msg.data:
        if pygame.mixer.music.get_busy():
            self.stop_event.set()
        if self.action_runing:
            self.interrupt_flag = True
            self.stop()
            self.pubSix_Arm(self.init_joints)
```

The `play_audio` function checks if `self.asr_detect.extern_wakeup` is set. If it is, the currently playing audio is stopped immediately.

```
    def play_audio(self,file_path: str) -> None:
        '''Play audio'''
        self.asr_detect.extern_wakeup.clear()
        with self.pygame_lock:
            pygame.mixer.init()
            pygame.mixer.music.load(file_path)
            pygame.mixer.music.play()
            while pygame.mixer.music.get_busy():
                if self.asr_detect.extern_wakeup.is_set():
                    pygame.mixer.music.stop()
                    self.asr_detect.extern_wakeup.clear()
                    break
                pygame.time.Clock().tick(10)
            pygame.mixer.quit()
```

6.3 Interrupting During Action

If the robot is woken up while performing an action, it will stop the current action and return to its initial posture. This can be divided into normal action interruption and interruption of actions with subprocesses.

6.3.1 Normal Action Interruption

- The robot's chassis movement is controlled by publishing velocity topics.
- Taking the dance action as an example: the chassis control function continuously checks the `self.interrupt_event` flag. If it is set, the chassis stops immediately.

```
    def dance(self):  # Dance
        actions = [
            {"linear_x": 0.6, "linear_y": 0.0, "angular_z": 0.0, "durationtime":
1.5},
            {"linear_x": -0.4, "linear_y": 0.0, "angular_z": 0.0,
"durationtime": 1.0},
            {"linear_x": 0.0, "linear_y": 0.3, "angular_z": 0.0, "durationtime":
1.0},
            {"linear_x": 0.0, "linear_y": -0.3, "angular_z": 0.0,
"durationtime": 1.0},
            {"linear_x": 0.0, "linear_y": 0.0, "angular_z": 0.6, "durationtime":
3.0},
            {"linear_x": 0.0, "linear_y": 0.0, "angular_z": -0.6,
"durationtime": 3.0},
        ]
        for action in actions:
            if self.interrupt_event.is_set():
                return None
            twist = Twist()
            twist.linear.x = action["linear_x"]
            twist.linear.y = action["linear_y"]
            twist.angular.z = action["angular_z"]
            self._execute_action(twist, durationtime=action["durationtime"])
        self.stop()
        return True
```

6.3.2 Interruption of Actions with Subprocesses

Actions like autonomous line following and object tracking require starting external programs in a subprocess. Here, we use autonomous line following as an example:

- Before the action is finished, it continuously checks if the `self.interrupt_event.is_set()` flag is set. If it is, an interruption has occurred, and the `kill_process_tree` function is called to terminate the subprocess.

```
    def follow_line(self,color) -> None:
        self.follow_line_future = Future()  # Reset Future object
        color = color.strip("'\"")  # Remove single and double quotes
        if color == 'red':
            target_color = float(1)
        elif color == 'green':
            target_color = float(2)
        elif color == 'blue':
            target_color = float(3)
```

```
        elif color == 'yellow':
            target_color = float(4)
        else:
            self.get_logger().info(Fore.RED+'color_sort:error'+Fore.RESET)
            return

        self.follow_line_process = subprocess.Popen(['ros2', 'run',
'multi_brains', 'follow_line','--ros-args','-p',f'colcor:={target_color}'])

        while not self.follow_line_future.done():
            if self.interrupt_event.is_set():
                kill_process_tree(self.follow_line_process.pid)
                self.stop()
                return None
            time.sleep(0.1)
        self.follow_line_clear_future = None
        kill_process_tree(self.follow_line_process.pid)
        return True
```